

Meeting Notes Summariser

A lightweight, local-first web app to upload meeting notes, PDFs, or audio files and chat with them using Retrieval-Augmented Generation (RAG). Built with a FastAPI server and a React + Vite + Tailwind client.

1. Introduction & Project Purpose

What Problem It Solves

After a meeting ends, important decisions, action items, and context get buried inside raw notes, PDFs, or hours of recorded audio. Reading through everything manually is slow, error-prone, and easy to procrastinate on.

The Meeting Notes Summariser lets you upload that content and immediately have a conversation with it. You ask questions in plain English — *"What were the action items from the product sync?"* or *"What did we decide about the Q3 budget?"* — and get AI-generated answers grounded directly in your actual documents, not in the model's general training knowledge.

Who It's For

- Engineers and PMs who attend many meetings and need to extract decisions quickly
- Anyone who records standups, interviews, or planning sessions and wants to query them later
- Teams that want a self-hosted, private alternative to cloud-based meeting intelligence tools

2. Prerequisites

Before running anything, make sure you have the following installed and available on your machine.

Requirement	Minimum Version	Purpose
Python	3.9+	Running the FastAPI backend server
Node.js	18+	Running the Vite/React frontend client
npm	Bundled with Node.js	Installing client-side dependencies
Google Gemini API key	—	Optional. Required only for audio transcription and LLM answer generation

3. Project Structure

The repository is split into two top-level directories: `server/` for the Python backend and `client/` for the React frontend. Below is a breakdown of every important file and what it does.

meeting-notes-summariser/

```
|
|
|— server/
|   |
|   |— main.py          # FastAPI app entrypoint. Creates the app instance,
|   |                   # registers routers, and runs startup logic.
|   |
|   |— requirements.txt # All Python dependencies (FastAPI, LangChain,
|   |                   # ChromaDB, pypdf, sentence-transformers, etc.)
|   |
|   |— .env             # Your local secrets (not committed to git).
|   |                   # Contains GOOGLE_API_KEY and CHROMA_PERSIST_DIRECTORY.
|   |
|   |— api/
|   |   |
|   |   |— routes.py    # Defines the three REST endpoints: /api/upload,
|   |   |               # /api/chat, and /api/history/{session_id}.
|   |   |
|   |   |— services/
|   |   |   |
|   |   |   |— llm.py    # The core AI logic. Contains add_document() for
|   |   |   |           # chunking and embedding, query_summarizer() for RAG
|   |   |   |           # retrieval, and transcribe_audio_file() for Gemini
|   |   |   |           # audio transcription.
|   |   |
|   |   |— database/
|   |   |   |
|   |   |   |— models.py # SQLite table definition for chat_messages.
|   |   |   |           # Handles reading and writing conversation history.
|   |
|
|— client/
|   |
|   |— index.html       # The single HTML file Vite serves. The React app
|   |                   # mounts into the <div id="root"> here.
|   |
|   |— tailwind.config.js # Tailwind configuration. The `content` array tells
|   |                   # Tailwind which files to scan for class names so
|   |                   # unused styles get purged in production builds.
|   |
|   |— postcss.config.js # PostCSS config. Vite reads this automatically and
|   |                   # uses it to process Tailwind directives.
|   |
|   |— src/
|   |   |
|   |   |— main.tsx      # React entry point. Renders <App /> into #root.
|   |   |
|   |   |— App.tsx       # Root component. Manages session state, renders the
|   |   |               # sidebar, the upload panel, and the chat interface.
|   |   |
|   |   |— api.ts        # All HTTP calls to the backend in one place.
|   |   |               # Functions for uploadFile(), sendMessage(), and
```

```

| | | # loadHistory().
| | |
| | | └─ index.css # Contains the three Tailwind directives:
| | | # @tailwind base, components, and utilities.

```

4. Architecture Overview

The system has three distinct layers that each own a specific responsibility.

4.1 Client (Frontend)

The frontend is a single-page React application built with Vite and styled with Tailwind CSS. TypeScript is used throughout for type safety.

The UI is organised around two main panels:

- **Add Context panel:** Where you upload files or paste raw text. This triggers a POST `/api/upload` request and shows a confirmation of how many chunks were added to the vector store.
- **Chat panel:** A standard message thread UI where you type questions and see AI responses. Each response optionally shows the source chunks that were used to generate the answer.

The client stores session metadata (session IDs and display names) in the browser's `localStorage`. This means sessions survive page refreshes. However, the full message content lives in SQLite on the server — `localStorage` only holds enough information for the client to know which session to request history for.

4.2 Server (Backend)

The backend is a FastAPI application that exposes all functionality through a small REST API. It handles three responsibilities:

- **File ingestion:** Receives uploaded files, extracts text based on file type, splits the text into chunks, generates embeddings, and stores them in ChromaDB.
- **RAG querying:** On each chat message, retrieves the most semantically relevant chunks from ChromaDB, assembles them into a context string, and forwards the context plus the question to Gemini for answer generation.
- **History persistence:** Logs every user message and assistant reply to SQLite so sessions can be reloaded later.

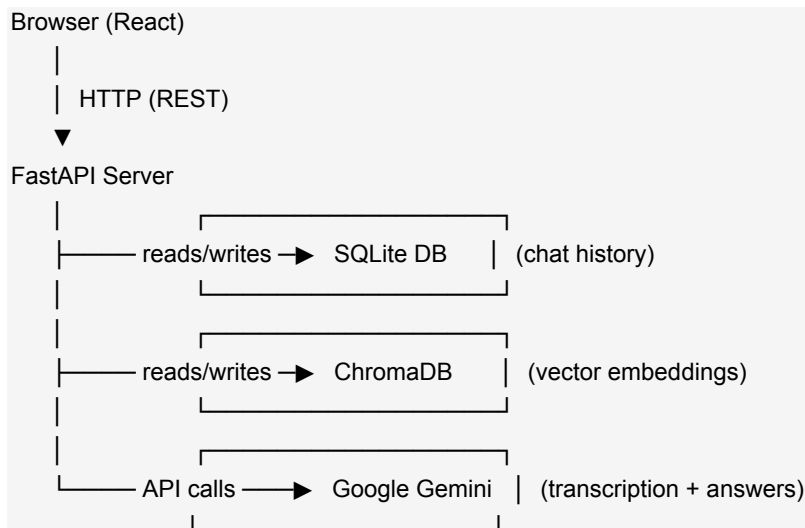
4.3 Data Stores

The application uses two persistent stores and one browser-side store:

Store	Technology	What it holds	Lives where
Vector store	ChromaDB	Embeddings of all document chunks, plus metadata (source filename, session ID)	On disk in the <code>CHROMA_PERSIST_DIRECTORY</code> folder
Message	SQLite	Every chat	On disk as a local <code>.db</code> file

store		message with its role, content, session ID, and timestamp	
Session metadata	localStorage	Session IDs and display names only	In the user's browser

4.4 How the Three Layers Talk to Each Other



The browser never talks to ChromaDB, SQLite, or Gemini directly. Everything goes through the FastAPI server, which acts as the single source of truth for all AI and storage operations.

5. End-to-End Data Flow

5.1 Upload Flow

When you upload content, the server handles it differently depending on the file type.

Plain text (.txt or pasted text)

The text string is passed directly to `add_document()` with no preprocessing. This is the fastest path.

PDF (.pdf)

The server opens the file with `pypdf.PdfReader` and iterates over every page, extracting its text content. All pages are concatenated into a single string, which is then passed to `add_document()`. Note that this method works well for text-based PDFs but will produce poor results for scanned PDFs that are image-only (no OCR is applied).

Audio (.mp3, .wav, .m4a, .flac)

The server saves the uploaded file to a temporary location on disk, then calls `transcribe_audio_file()`. This function sends the audio to Google Gemini's transcription API and returns a plain text transcript. The transcript is then passed to `add_document()` exactly like any other text input. This step requires a valid `GOOGLE_API_KEY`.

5.2 Chunking and Embedding Deep-Dive

Once text is available (regardless of its source), it goes through `add_document()`. This function does the following:

Step 1 — Split into chunks

`RecursiveCharacterTextSplitter` breaks the text into smaller pieces. The defaults are:

- `chunk_size` = 1000 characters per chunk
- `chunk_overlap` = 200 characters of overlap between adjacent chunks

The overlap is important. Without it, a sentence that happens to sit on the boundary between two chunks would be split in half. With 200 characters of overlap, context is shared between neighbouring chunks, so retrieval is less brittle.

`RecursiveCharacterTextSplitter` tries to split on natural boundaries first — paragraphs, then sentences, then words, then individual characters — so chunks tend to end at sensible points rather than mid-word.

Step 2 — Attach metadata

Each chunk is wrapped in a LangChain Document object that carries two metadata fields:

- `source`: the original filename (e.g. `standup-2024-07-01.pdf`)
- `session_id`: the session this upload belongs to

The `session_id` metadata is what makes session-scoped retrieval possible later.

Step 3 — Generate embeddings

Each chunk is passed through the `sentence-transformers/all-MiniLM-L6-v2` model, which runs locally via `HuggingFaceEmbeddings`. This model converts the text of each chunk into a vector of 384 floating-point numbers that encodes the semantic meaning of the text. Similar sentences produce similar vectors; unrelated sentences produce very different vectors.

The model runs entirely on your machine. No text is sent anywhere during the embedding step.

Step 4 — Store in ChromaDB

The chunks and their embeddings are added to ChromaDB with `vectorstore.add_documents(chunks)`. ChromaDB writes them to the `persist_directory` on disk. Because the index is persisted, you can restart the server and all previously uploaded documents are still searchable.

5.3 RAG Query Flow

When you send a chat message, the following sequence runs:

1. **Log user message** — the message is written to the `chat_messages` table in SQLite with `role = "user"`.
2. **Embed the question** — the same `all-MiniLM-L6-v2` model converts your question into a 384-dimensional vector.
3. **Similarity search** — ChromaDB compares your question's vector against all stored chunk vectors, filtered to only the current `session_id`. It returns the top 5 most semantically similar chunks (configurable via `k=5`).
4. **Build context** — the text of the 5 retrieved chunks is concatenated into a single context string.

5. **Construct the prompt** — a system prompt tells Gemini to act as an expert meeting summariser and to answer only based on the provided context. The context string and your original question are included.
6. **Call Gemini** — ChatGoogleGenerativeAI sends the prompt to Google's API and returns a generated answer.
7. **Return to client** — the server sends back { answer, sources } where sources contains the raw text of the retrieved chunks so the client can display them.
8. **Log assistant reply** — the answer is written to SQLite with role = "assistant" so the conversation is preserved.

6. API Reference

POST /api/upload

Ingests a document or audio file into the vector store.

Request format: multipart/form-data

Field	Type	Required	Description
file	File	No*	A .txt, .pdf, .mp3, .wav, .m4a, or .flac file
text	String	No*	Raw text to ingest directly
session_id	String	No	The session to associate this content with. Auto-generated if omitted.

*Either file or text must be provided, not both.

Response:

```
{ "chunks_added": 14, "session_id": "abc-123" }
```

chunks_added tells you how many chunks were created from the upload. A typical one-hour meeting transcript might produce 40–80 chunks depending on length.

POST /api/chat

Sends a user message and returns an AI-generated answer with source chunks.

Request format: application/json

```
{ "session_id": "abc-123", "message": "What were the action items from the meeting?" }
```

Response:

```
{ "answer": "The action items were: (1) Alice to send the revised budget by Friday...", "sources": [ { "content": "Alice will send the revised budget by end of week...", "metadata": { "source": "standup.pdf", "session_id": "abc-123" } } ] }
```

sources is an array of the raw chunks that were retrieved from ChromaDB and used to generate the answer. You can use this to verify which parts of your documents the answer is based on.

GET /api/history/{session_id}

Returns the full conversation history for a session in chronological order.

Response:

```
[ { "role": "user", "content": "What were the action items?", "created_at": "2024-07-01T10:32:00Z" },  
  { "role": "assistant", "content": "The action items were...", "created_at": "2024-07-01T10:32:03Z" } ]
```

This endpoint is called by the client on page load to restore a previous session's messages into the chat UI.

7. Configuration and Environment Variables

Create a file called `.env` inside the `server/` directory. This file is listed in `.gitignore` and must never be committed to version control.

```
GOOGLE_API_KEY=your_gemini_api_key_here
```

```
CHROMA_PERSIST_DIRECTORY=./vector_store
```

Variable	Required	Default	Description
GOOGLE_API_KEY	Only for Gemini features	—	Your Google Generative AI API key. Used for audio transcription via <code>transcribe_audio_file()</code> and for answer generation via <code>ChatGoogleGenerativeAI</code> . If omitted, both features will fail with an authentication error.
CHROMA_PERSIST_DIRECTORY	No	<code>./vector_store</code>	Path to the folder where ChromaDB writes its index files. If the folder already exists and contains a previous index, it will be reused and new documents will be added to it.

Tips:

- Use an absolute path for `CHROMA_PERSIST_DIRECTORY` if you run the server from multiple working directories, to avoid accidentally creating multiple index folders.
- If you rotate your `GOOGLE_API_KEY`, restart the server after updating `.env` — environment variables are read at startup.

8. How to Run Locally

Server (Python / FastAPI)

```
# Navigate to the server directory  
cd server
```

```
# Create a virtual environment to isolate Python dependencies  
python -m venv venv
```

```
# Activate the virtual environment
```

```
# On macOS/Linux:
source venv/bin/activate
# On Windows:
venv\Scripts\activate
```

```
# Install all Python dependencies
pip install -r requirements.txt
```

```
# Create your .env file (see Configuration section above)
cp .env.example .env
# Then edit .env and fill in your values
```

```
# Start the development server with hot-reload
uvicorn main:app --reload --port 8000
```

The API will be available at <http://localhost:8000>. You can explore the auto-generated API docs at <http://localhost:8000/docs>.

Client (Vite / React / Tailwind)

```
# In a separate terminal, navigate to the client directory
cd client
```

```
# Install Node.js dependencies
npm install
```

```
# Start the Vite development server
npm run dev
```

Vite will print a local URL — usually <http://localhost:5173>. Open it in your browser.

9. Database Schema

Chat history is stored in a SQLite database managed by `server/database/models.py`. SQLite requires no separate installation — Python includes it in its standard library.

chat_messages Table

Column	Type	Description
id	INTEGER PRIMARY KEY	Auto-incrementing row ID. Not exposed to the client.
session_id	TEXT	Identifies which conversation this message belongs to. Used to filter history in <code>GET /api/history/{session_id}</code> .
role	TEXT	Either "user" or "assistant". Determines which side of the chat UI the message appears on.
content	TEXT	The full text of the message — either the user's question or the AI's generated answer.

created_at	DATETIME	UTC timestamp of when the message was stored. Used to return history in chronological order.
------------	----------	--

10. Security and Privacy

What Stays Local

Data	Where it goes
Document text and embeddings	Stored on disk in ChromaDB. Never leaves your machine.
Chat history	Stored in SQLite on your machine. Never leaves your machine.
Embedding computation	Runs locally using the HuggingFace model. No network calls.

What Goes to Google

Data	When
Audio file contents	When you upload an audio file and GOOGLE_API_KEY is set — sent to Gemini for transcription.
Retrieved chunk text + your question	On every chat message — sent to Gemini to generate an answer.

If you are working with sensitive meeting content, consider running a fully local LLM (e.g. via Ollama) so no data leaves your machine at any point. See the "Swap the LLM Provider" section above.

API Key Best Practices

- Store your GOOGLE_API_KEY only in server/.env, which is excluded from git via .gitignore.
- Never hardcode the key in source files.
- If you accidentally commit the key, rotate it immediately in Google AI Studio and revoke the old one.
- Consider restricting the API key to only the Gemini APIs it needs, using Google Cloud's key restriction settings.

11. Limitations

localStorage fragility: The client stores only session IDs and names in the browser — not message content. If a user clears their browser storage (or uses a different browser or device), they lose the ability to navigate back to old sessions, even though all the messages still exist in SQLite on the server. A future improvement would be to list all sessions via a server-side API endpoint so the UI doesn't depend on localStorage to discover sessions.

Retrieval quality depends on chunking: The default chunk_size=1000 and chunk_overlap=200 are reasonable starting points but are not optimal for every document type. Dense technical documents may benefit from smaller chunks (e.g. 500 characters). Long narrative transcripts may benefit from larger chunks (e.g. 1500 characters). Experimenting with these values and evaluating retrieval quality with representative questions is the highest-leverage way to improve answer quality.

Simple prompt engineering: The current system prompt is a single instruction telling Gemini to act as a meeting summariser. There is significant headroom for improvement: adding few-shot examples, instructing the model to cite chunk sources explicitly in its answer, asking it to flag when the context is insufficient to answer the question confidently, or formatting output as structured JSON for action items versus free text for summaries.

No OCR for scanned PDFs: The PDF extraction path uses pypdf, which reads the text layer of a PDF directly. Scanned documents that are images of text have no text layer, so pypdf will return empty or near-empty content for them. Adding an OCR step (e.g. using pytesseract or a cloud OCR API) before passing content to `add_document()` would address this.

Single-user

assumption

The current architecture assumes a single local user. There is no authentication, no per-user access control, and no isolation between different users' data if the server were exposed to a network. Do not expose this server to the public internet without adding an authentication layer.